

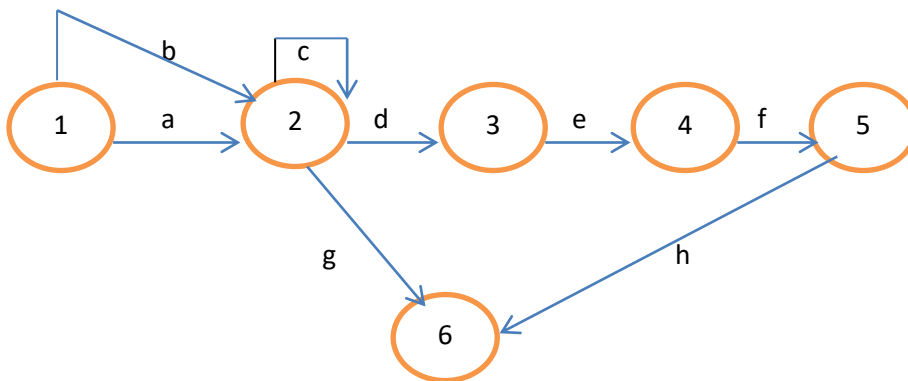
State based or Graph based Testing

It is very useful in situations where

1. The product under test is a language processor.
2. Work flow modelling, where depending on the current state and appropriate combinations of input variables, specific workflows are carried out, resulting in new output and new state.
3. Data flow modelling

Consider an application that is required to validate a number according to the following simple rules.

- A number can start with an optional sign
- The optional sign can be followed by any number of digits
- The digits can be optionally followed by a decimal point
- If there is a decimal point, then there should be two digits after the decimal
- Any number, should be terminated by a blank.



State Transition Graph for validating a number

Where

1. a denotes digit
2. b denotes + or –
3. c denotes Digit
4. d denotes decimal point
5. e denotes digit

6. f denotes digit
7. g, h denotes blank

State transition graph can be converted to a state transition table.

Current state	Input	Next State
1	Digit	2
1	+	2
1	-	2
2	Digit	2
2	Blank	6
2	Decimal point	3
3	Digit	4
4	Digit	5
5	Blank	6

STT can be used to derive test cases to test valid and invalid numbers.

Valid test cases can be generated by

1. Start from the start state
2. Choose a path that leads to the next state
3. If you encounter an invalid input in a given state, generate an error condition test case
4. Repeat the process till you reach the final state.

Test Case Id	Input	Expected Output
1.	-123.45	Valid Input – Start to end traversed successfully
2.	000000	Valid
3.	+345.-25	Invalid Input
4.	Abc.ef	Invalid Input
5.	-23.3	Invalid Input
6.	345711	Valid Input
7.	- -3.45	Invalid Input
8.	10001.35	Valid Input
9.	Abc.35	Invalid Input
10.	+	Invalid Input

